Transformers in Large Language Models (LLMs)

Benoît Legat

☐ Full Width Mode ☐ Present Mode

Table of Contents

Large Language Models (LLMs)

Pre-transformers approaches

Attention is all you need

Decoder-only transformer

Encoder-decoder transformer

Energy consumption of LLMs

Large Language Models (LLMs)

References

- Recurrent neural networks : [GBC16; Chapter 10] and Section 4.7 of The Elements of Differentiable Programming book
- Transformers : [VSP+17] and Section 4.8 of The Elements of Differentiable Programming book
- Neural Networks: Zero to Hero by Andrej Karpathy

[GBC16] I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning* (MIT Press, 2016). Accessed on Aug 28, 2024.
[VSP+17] A. Vaswani, N. Shazeer, N. Parmar *et al.* *Attention Is All You Need*. In: *Advances in Neural Information Processing Systems*, Vol. 30 (Curran Associates, Inc., 2017). Accessed on Oct 11, 2024.

Autoregressive Models

Given a sequence of n_{ctx} past vectors $x_{-1}, \dots, x_{-n_{\text{ctx}}} \in \mathbb{R}^n$, "predict" the next ones. Key idea : *receding horizon*:

$$\begin{aligned} & p(x_0, x_1 | x_{-1}, \dots, x_{-n_{\text{ctx}}}) \\ & = p(x_0 | x_{-1}, \dots, x_{-n_{\text{ctx}}}) p(x_1 | x_0, x_{-1}, \dots, x_{-n_{\text{ctx}}+1}, \boldsymbol{x_{-n_{\text{ctx}}}}) \\ & \approx p(x_0 | x_{-1}, \dots, x_{-n_{\text{ctx}}}) p(x_1 | x_0, x_{-1}, \dots, x_{-n_{\text{ctx}}+1}) \end{aligned}$$

- **Model** : Probability of next vector $\hat{p}(x_0 | X)$ where X concatenates $x_{-1}, \dots, x_{-n_{\text{ctx}}}$.
- **Loss** : Cross-entropy : $\mathcal{L}_{\hat{p}}(X) \triangleq H(p, \hat{p}) = -\mathbf{E}_p[\log(\hat{p})] = -\sum_{x_0} p(x_0|X) \log(\hat{p}(x_0|X))$
- Particular case for $p(x_0|X) = \delta_y$: $\mathcal{L}_{\hat{p}}(X) = -\log(\hat{p}(y|X))$

What about Language Models ?

Given "past text", predict the "following text". How to turn text into vectors of $\mathbb{R}^n$?

Text to vectors : step 1 → tokenization

Why not encode each letter ?

- **Idea** : Turn each letter into its one-hot encoding in $\mathbb{R}^{26}$.

- **Issue** : The "past text" only has n_{ctx} characters so n_{ctx} must be **large** but transformers have a complexity **quadratic** in n_{ctx}!
- **Practical details** : Text is encoded with UTF-8 so each character is encoded into 1 to 4 bytes. We encode each byte to a vector in $\mathbb{R}^{256}$ but care must be taken not to generate invalid UTF-8.

Why not encode each word ?

- **Idea** : Turn each word into its one-hot encoding in $\mathbb{R}^n$. The value of n is the number of words. Depending on the language (source):

Language	French	English	Dutch	German
n	408,078	350,000	350,000	200,000

- **Issue** : The value of n is **too large**. We cannot trust the words of languages to be a tokenization that optimally compresses text for our dataset.

Byte Pair Encoding

Byte Pair Encoding algorithm [SHB16] greedily merges the most frequent pair of tokens over the dataset into a new token. Most used implementations are `SentencePiece` [KR18] and `tiktoken` (play with it here). For instance, on this example, the pair `('a', 'a')` is the most frequent so we substitute it by a new token, say `'Z'` :

```
▸ Dict(('d', 'a') ⇒ 1, ('a', 'b') ⇒ 2, ('b', 'd') ⇒ 1, ('b', 'a') ⇒ 1, ('a', 'a') ⇒ 4, ('
```

```
1 pair_stats("aaabdaaabac")
```

```
iter_1 = ▸ BPE("ZabdZabac", Dict(('a', 'a') ⇒ 'Z'))
```

```
1 iter_1 = new_token("aaabdaaabac")
```

```
iter_2 = ▸ BPE("ZYdZYac", Dict(('a', 'b') ⇒ 'Y', ('a', 'a') ⇒ 'Z'))
```

```
1 iter_2 = new_token(iter_1)
```

Note that the new tokens can also be part of the most frequence pair!

```
iter_3 = ▸ BPE("XdXac", Dict(('a', 'b') ⇒ 'Y', ('Z', 'Y') ⇒ 'X', ('a', 'a') ⇒ 'Z'))
```

```
1 iter_3 = new_token(iter_2)
```

[SHB16] R. Sennrich, B. Haddow and A. Birch. *Neural Machine Translation of Rare Words with Subword Units* (Jun 2016), arXiv:1508.07909. Accessed on Oct 23, 2024.
[KR18] T. Kudo and J. Richardson. *SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing* (Aug 2018), arXiv:1808.06226. Accessed on Oct 23, 2024.

Increasing length of "past text"

> **Challenging tradeoff**: Encode text to **increase** length of "past text" while keeping n_{ctx} and n **small** enough.

Length of "past text" increases with vocabulary size n_{voc} and context window n_{ctx}.

Name	Ref	n_{voc}	n_{ctx}	Tokenizer
GPT-2	[RWCL19]	50k	1024	tiktoken
GPT-3	[BMRS20]	50k	2048	tiktoken
GPT-3.5		100k	4096	tiktoken
GPT-4		100k	32k	tiktoken
GPT-4o		200k	128k	tiktoken
Gemini-1	[TABA24]	256k	10M	SentencePiece
Gemini-1.5	[TGLB24]	256k	10M	SentencePiece
Gemma	[TMHD24]	256k	8192	SentencePiece
Gemma-2	[TRPS24]	256k	8192	SentencePiece
Llama-2	[TMSA23]	32k	4k	SentencePiece
Llama-3		128k	8k	tiktoken
Llama-3.1		128k	128k	tiktoken
Llama-3.2		128k	128k	tiktoken
MegaByte	[YSFA23]	256	8192	Bytes

[TGL+24] G. Team, P. Georgiev, V. I. Lei *et al.* *Gemini 1.5: Unlocking Multimodal Understanding across Millions of Tokens of Context* (Aug 2024), arXiv:2403.05530. Accessed on Nov 11, 2024.
[TAB+24] G. Team, R. Anil, S. Borgeaud *et al.* *Gemini: A Family of Highly Capable Multimodal Models* (Jun 2024), arXiv:2312.11805. Accessed on Nov 11, 2024.
[TMH+24] G. Team, T. Mesnard, C. Hardin *et al.* *Gemma: Open Models Based on Gemini Research and Technology* (Apr 2024), arXiv:2403.08295. Accessed on Nov 11, 2024.
[TRP+24] G. Team, M. Riviere, S. Pathak *et al.* *Gemma 2: Improving Open Language Models at a Practical Size* (Oct 2024), arXiv:2408.00118. Accessed on Nov 11, 2024.
[SHB16] R. Sennrich, B. Haddow and A. Birch. *Neural Machine Translation of Rare Words with Subword Units* (Jun 2016), arXiv:1508.07909. Accessed on Oct 23, 2024.
[RWC+19] A. Radford, J. Wu, R. Child *et al.* *Language Models Are Unsupervised Multitask Learners* (2019). Accessed on Oct 23,

2024.
[BMR+20] T. B. Brown, B. Mann, N. Ryder *et al.* *Language Models Are Few-Shot Learners* (Jul 2020), arXiv:2005.14165. Accessed on Nov 11, 2024.
[TMS+23] H. Touvron, L. Martin, K. Stone *et al.* *Llama 2: Open Foundation and Fine-Tuned Chat Models* (Jul 2023), arXiv:2307.09288. Accessed on Oct 23, 2024.
[YSF+23] L. Yu, D. Simig, C. Flaherty *et al.* *MEGABYTE: Predicting Million-byte Sequences with Multiscale Transformers* (May 2023), arXiv:2305.07185. Accessed on Oct 23, 2024.

Text to vectors : step 2 → embedding

Consider one-hot encoding with vocabulary size n_{voc} and a *bigram model*

$$\hat{p}(x_0|x_{-1}) = \mathbf{softmax}(W_d \tanh(\cdots \tanh(W_1 x_{-1}) \cdots)$$

The matrix W_d has n_{voc} rows and W_1 has n_{voc} columns → issue if n_{voc} is large

Embedding : Use vectors $c_1, \ldots, c_{n_{\text{voc}}} \in \mathbb{R}^{d_{\text{emb}}}$ with *embedding size* (aka *hidden size*) $d_{\text{emb}} \ll n_{\text{voc}}$.

Equivalently, we still use one-hot encoding but we add an encoder $C \in \mathbb{R}^{d_{\text{emb}} \times n_{\text{voc}}}$ and decoder $D \in \mathbb{R}^{n_{\text{voc}} \times d_{\text{emb}}}$

$$\hat{p}(x_0|x_{-1}) = \mathbf{softmax}(DW_d \tanh(\cdots \tanh(W_1 C x_{-1}) \cdots)$$

▸ **What difference do you expect with respect to the previous model ?**

Forcing $D = C^\top$ appears to work well in practice [PW17], this is what is used in [VSP+17].

▸ **Why don't we use $D = C^{-1}$?**

[PW17] O. Press and L. Wolf. *Using the Output Embedding to Improve Language Models* (Feb 2017), arXiv:1608.05859. Accessed on Nov 11, 2024.
[VSP+17] A. Vaswani, N. Shazeer, N. Parmar *et al.* *Attention Is All You Need*. In: *Advances in Neural Information Processing Systems*, Vol. 30 (Curran Associates, Inc., 2017). Accessed on Oct 11, 2024.

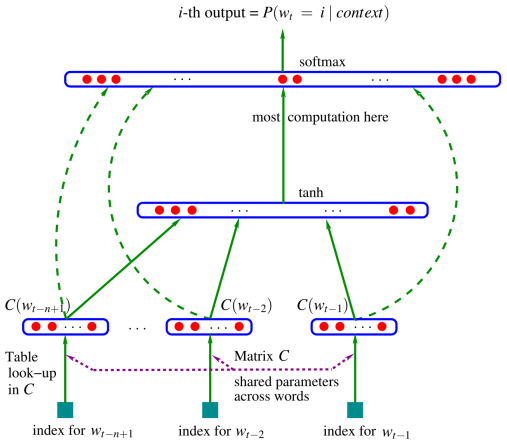
Pre-transformers approaches

Shared embedding

With $n_{\text{ctx}} > 1$, the encoder C is shared by all tokens: See for instance the network below taken from [BDV00; Figure 1], the first popular application of neural nets for languages:

$$\hat{p}(x_0|x_{-1}, \ldots, x_{-n_{\text{ctx}}}) =$$

$$\mathbf{softmax}(C^\top W_2 \tanh(W_1 \begin{bmatrix} Cx_{-1} \\ \vdots \\ Cx_{-n_{\text{ctx}}} \end{bmatrix}))$$



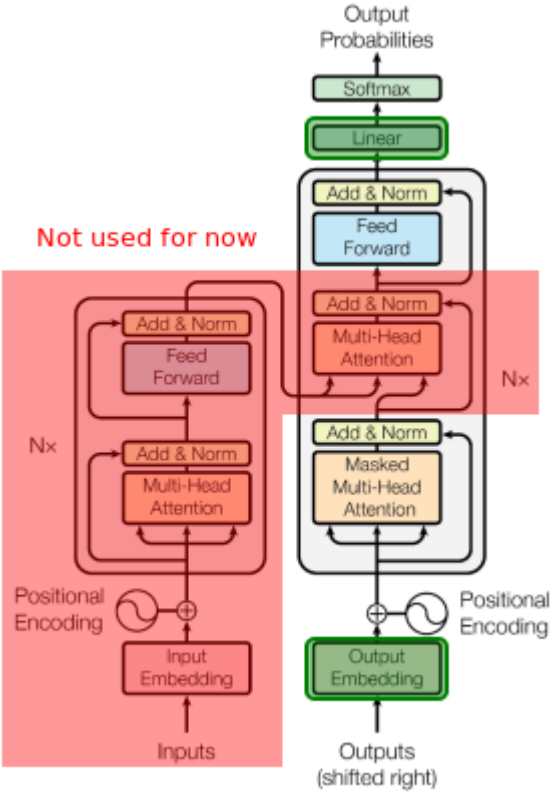
▶ **What are the number of columns of W_1 and number of rows of W_2 now ?**

[BDV00] Y. Bengio, R. Ducharme and P. Vincent. A Neural Probabilistic Language Model. In: *Advances in Neural Information Processing Systems*, Vol. 13 (MIT Press, 2000). Accessed on Oct 11, 2024.

Embedding sizes in LLMs

See the table below for the size of embeddings of large language models:

Name	Num params	Ref	n_{voc}	d_{emb}
GPT-2	1.5B	[RWCL19]	50k	768
Gemma	2B	[TMHD24]	256k	2048
Gemma	7B	[TMHD24]	256k	3072
Gemma-2	27B	[TRPS24]	256k	4608
Gemma-2	2B	[TRPS24]	256k	2304
Gemma-2	9B	[TRPS24]	256k	3584
Llama-2	7B	[TMSA23]	32k	4096
base		[VSPU17]	37k	512
big		[VSPU17]	37k	1024



[TMH+24] G. Team, T. Mesnard, C. Hardin *et al.* *Gemma: Open Models Based on Gemini Research and Technology* (Apr 2024), arXiv:2403.08295. Accessed on Nov 11, 2024.

[TRP+24] G. Team, M. Riviere, S. Pathak *et al.* *Gemma 2: Improving Open Language Models at a Practical Size* (Oct 2024), arXiv:2408.00118. Accessed on Nov 11, 2024.

[RWC+19] A. Radford, J. Wu, R. Child *et al.* *Language Models Are Unsupervised Multitask Learners* (2019). Accessed on Oct 23, 2024.

[TMS+23] H. Touvron, L. Martin, K. Stone *et al.* *Llama 2: Open Foundation and Fine-Tuned Chat Models* (Jul 2023), arXiv:2307.09288. Accessed on Oct 23, 2024.

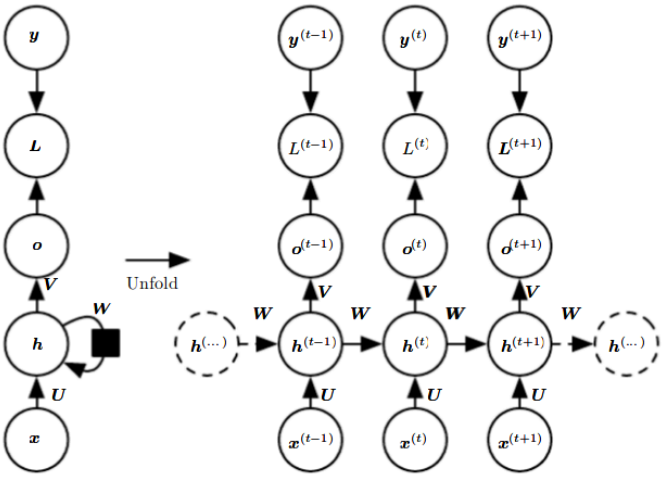
Recurrent neural networks (RNN)

$$
\begin{aligned}
h^{(t+1)} &= \tanh(Wh^{(t)} + Ux^{(t+1)} + b) \\
o^{(t)} &= Vh^{(t)} + c \\
\hat{y}^{(t)} &= \operatorname{softmax}(o^{(t)})
\end{aligned}
$$

Illustrated on the right [GBC16; Figure 10.3].

RNNs as language model showcased in [MKB+10].

Issue: Training time and space complexity is proportional to n_{ctx} and **cannot parallelize** to speed up.



[MKB+10] T. Mikolov, M. Karafiát, L. Burget *et al.* *Recurrent Neural Network Based Language Model*. In: *Proc. Interspeech 2010* (2010); pp. 1045–1048. Accessed on Nov 11, 2024.
[GBC16] I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning* (MIT Press, 2016). Accessed on Aug 28, 2024.

Extensions of RNNs

It's difficult to model long-term dependencies as their gradient either vanish or explodes exponentially (think of the power method) [GBC16; Section 10.7]

Gated extensions attempting to solve this issue [GBC16; Section 10.10]:

- Long short-term memory (LSTM) [Gra14]
- Gated recurrent unit (GRU) [CvMBB14]

Recently, Mamba suggests a solution to the complexity issue [GD24]. As it scales better with n_{ctx}, it is even suggested to get rid of the tokenizer : [WGYR24].

[CvMBB14] K. Cho, B. van Merrienboer, D. Bahdanau and Y. Bengio. *On the Properties of Neural Machine Translation: Encoder-Decoder Approaches* (Oct 2014), arXiv:1409.1259. Accessed on Nov 11, 2024.
[Gra14] A. Graves. *Generating Sequences With Recurrent Neural Networks* (Jun 2014), arXiv:1308.0850. Accessed on Nov 11, 2024.
[GBC16] I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning* (MIT Press, 2016). Accessed on Aug 28, 2024.
[GD24] A. Gu and T. Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces* (May 2024), arXiv:2312.00752. Accessed on Nov 11, 2024.
[WGYR24] J. Wang, T. Gangavarapu, J. N. Yan and A. M. Rush. *MambaByte: Token-free Selective State Space Model* (Aug 2024), arXiv:2401.13660. Accessed on Nov 11, 2024.

Attention is all you need

Numerical dictionary

What would a numerical dictionary look like ? Consider keys $k_i \in \mathbb{R}^{d_k}$ and values $v_i \in \mathbb{R}^{d_v}$. Given a query $q \in \mathbb{R}^{d_k}$,

```
dict = ▸Dict([0, 1] ⇒ [-1, 1], [1, 0] ⇒ [1, 1])
```

```
dict = Dict([1, 0] => [1, 1], [0, 1] => [-1, 1])
```

```
▸[1, 1]
```

```
dict[[1, 0]]
```

```
numerical_lookup (generic function with 1 method)
```

```
function numerical_lookup(dict, query)
    _, i = findmax([dot(query, key) for key in keys(dict)])
    return collect(values(dict))[i]
end
```

```
▸[1, 1]
```

```
numerical_lookup(dict, [0.8, 0.2])
```

Attention head

Attention head provides a differentiable numerical dictionary [BCB16]

$$\alpha = \mathrm{softmax}(\langle q, k_1 \rangle, \ldots, \langle q, k_{n_{\mathrm{ctx}}} \rangle) \qquad \mathrm{Attention}(q, k, v) = \sum_{i=1}^{n_{\mathrm{ctx}}} \alpha_i v_i$$

```
softmax (generic function with 1 method)
```

```
function softmax(x)
    y = exp.(x)
    return y / sum(y)
end
```

```
softmax_lookup (generic function with 1 method)
```

```
function softmax_lookup(dict, query)
    ks = keys(dict)
    α = softmax([dot(query, key) for key in keys(dict)])
    @show α
    return sum(α * value for (α, value) in zip(α, values(dict)))
end
```

```
▸ [0.291313, 1.0]
```

```
softmax_lookup(dict, [0.8, 0.2])
```

```
α = [0.3543436937742045, 0.6456563062257954]
```

[BCB16] D. Bahdanau, K. Cho and Y. Bengio. *Neural Machine Translation by Jointly Learning to Align and Translate* (May 2016), arXiv:1409.0473. Accessed on Oct 23, 2024.

Matrix form of attention

$$Q = \begin{bmatrix} q_1 & \cdots & q_{n_{\text{ctx}}} \end{bmatrix} \qquad K = \begin{bmatrix} k_1 & \cdots & k_{n_{\text{ctx}}} \end{bmatrix} \qquad K^\top Q = \begin{bmatrix} \langle k_1, q_1 \rangle & \cdots & \langle k_1, q_{n_{\text{ctx}}} \rangle \\ \vdots & \ddots & \vdots \\ \langle k_{n_{\text{ctx}}}, q_1 \rangle & \cdots & \langle k_{n_{\text{ctx}}}, q_{n_{\text{ctx}}} \rangle \end{bmatrix}$$

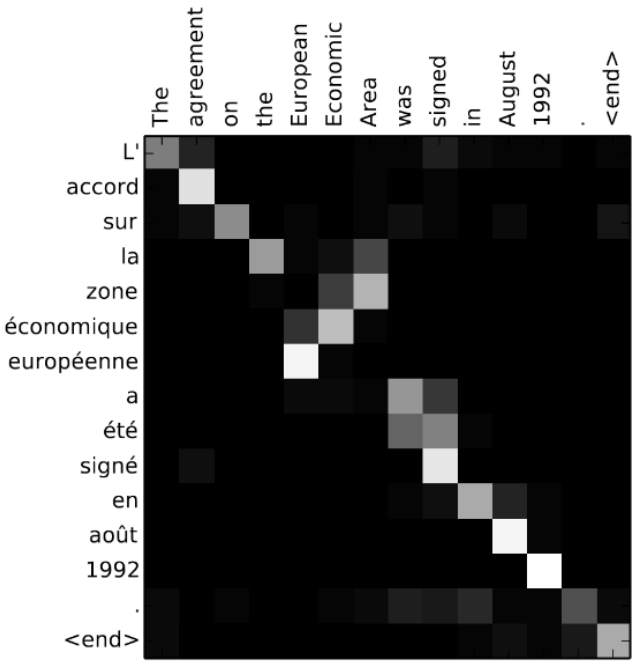
`softmax` is then applied to each **column**:

$$\mathbf{softmax}(K^\top Q / \sqrt{d_k})$$

Division by $\sqrt{d_k}$ scales the input of softmax to preferable regions [VSP+17; Secton 3.2.1].

Illustrated on the right from [BCB16; Figure 3(a)].

$$\mathbf{Attention}(V, K, Q) = V\mathbf{softmax}(K^\top Q / \sqrt{d_k})$$



[BCB16] D. Bahdanau, K. Cho and Y. Bengio. *Neural Machine Translation by Jointly Learning to Align and Translate* (May 2016), arXiv:1409.0473. Accessed on Oct 23, 2024.
[VSP+17] A. Vaswani, N. Shazeer, N. Parmar *et al.* *Attention Is All You Need*. In: *Advances in Neural Information Processing Systems*, Vol. 30 (Curran Associates, Inc., 2017). Accessed on Oct 11, 2024.

Masked Attention

💡 **Key idea** In the model for $\hat{p}(x_0|x_{-1},\dots,x_{-n_{\text{ctx}}})$, incorporate sub-models

$$\begin{aligned}&\bar{p}(x_0|x_{-1},\dots,x_{-n_{\text{ctx}}})\\&\bar{p}(x_{-1}|x_{-2},\dots,x_{-n_{\text{ctx}}})\\&\qquad\vdots\\&\bar{p}(x_{-n_{\text{ctx}}+1}|x_{-n_{\text{ctx}}}).\end{aligned}$$

Mask prevents $\hat{p}$ to look at inputs in the future:

$$M=\begin{bmatrix}0&0&\cdots&0\\-\infty&0&\ddots&\vdots\\\vdots&\ddots&\ddots&0\\-\infty&\cdots&-\infty&0\end{bmatrix}$$

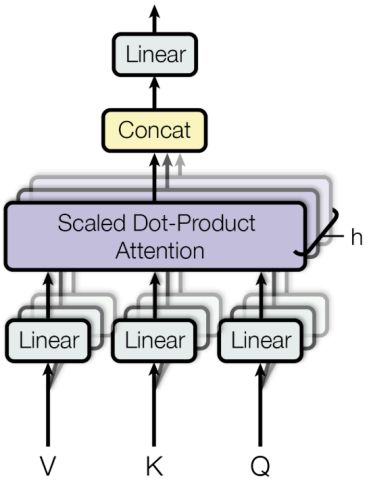
$$\text{Masked-Attention}(V,K,Q)=V\text{softmax}(M+K^\top Q/\sqrt{d_k})$$

Multi-Head Attention

Heads focus on different aspects. Their outputs are **combined** with $W^O\in\mathbb{R}^{d_{\text{emb}}\times hd_v}$:

$$\begin{aligned}\text{head}_j&=\text{Attention}(W_j^VV,W_j^KK,W_j^QQ)\\\text{MultiHead}(V,K,Q)&=W^O\text{vcat}(\text{head}_1,\dots,\text{head}_h)\end{aligned}$$

See [VSP+17; Figure 2] on the right.



Similarly, in the masked case:

$$\begin{aligned}\text{head}_j&=\text{Masked-Attention}(W_j^VV,W_j^KK,W_j^QQ)\\\text{Masked-MultiHead}(V,K,Q)&=W^O\text{vcat}(\text{head}_1,\dots,\text{head}_h)\end{aligned}$$

▸ **Is W^O needed if $h=1$?**

Further readings

This dictionary viewpoint can also be view as a *memory store* that recalls by similarity, called *associate memory*. See this blog post for more details.

Decoder-only transformer

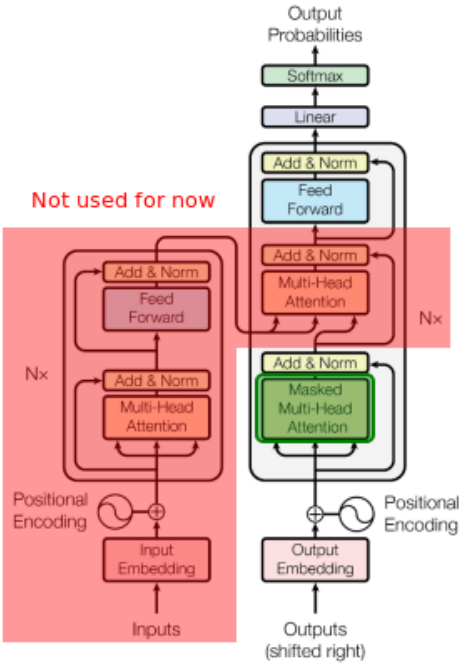
Self-Attention

Self-Attention with embedding C is:

$$\textbf{Masked-MultiHead}(CX, CX, CX)$$

The embedding vectors CX take then different projections for value, key, query and also for different heads!

$$\textbf{head}_j = \textbf{Masked-Attention}(W_j^V CX, W_j^K CX, W_j^Q CX)$$



▶ **Is the order between the tokens taken into account by the model ?**

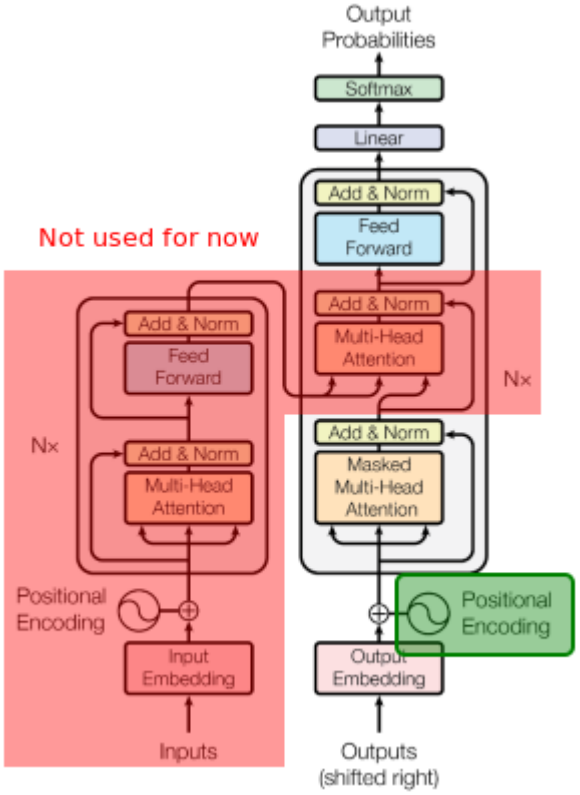
Positional encoding

Cannot sum $Cx_i + e_i$ with one-hot encoding $e_i \in \mathbb{R}^{n_{\text{ctx}}}$ as the dimension of Cx_i is $\mathbb{R}^{d_{\text{emb}}}$.

So we also add a positional embedding P : $Cx_i + Pe_i = Cx_i + p_i$.

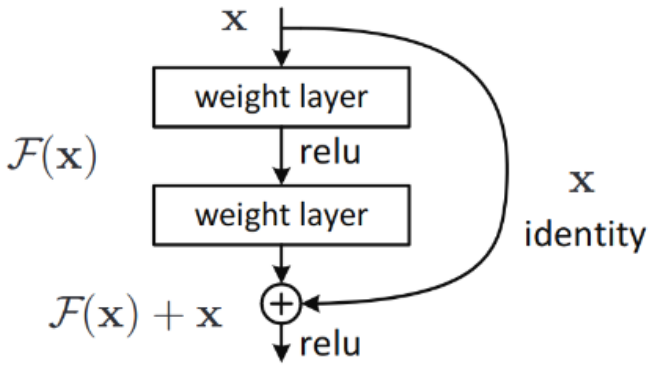
With Self-Attention:

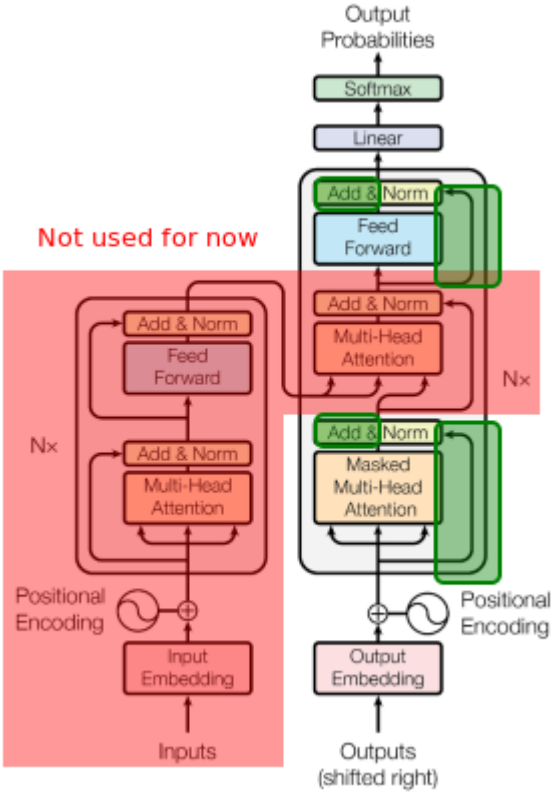
$$\textbf{Self-MultiHead}(CX + P, CX + P, CX + P)$$



Residual connection

Residual connection [HZRS15]





[HZRS15] K. He, X. Zhang, S. Ren and J. Sun. *Deep Residual Learning for Image Recognition* (Dec 2015), arXiv:1512.03385. Accessed on Nov 12, 2024.

Layer normalization

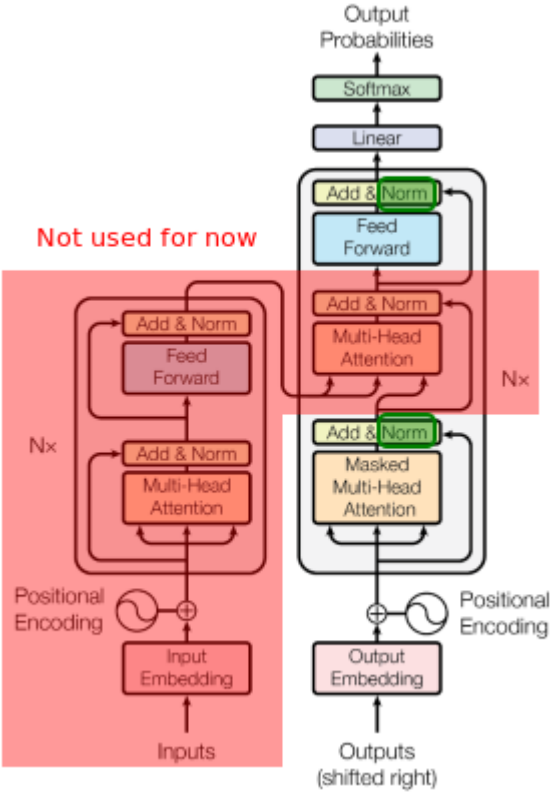
Norm of gradient increases exponentially with depth. Issue for deep neural net. Consider output

$$\begin{bmatrix} y_{1,1} & \cdots & y_{1,d_{\text{emb}}} \\ \vdots & \ddots & \vdots \\ y_{d_{\text{batch}},1} & \cdots & y_{d_{\text{batch}},d_{\text{emb}}} \end{bmatrix}$$

Normalization : $y_{i,j} \mapsto g(y_{i,j} - \mu_{i,j})/\sigma_{i,j}$ for gain g, mean μ and standard deviation σ.

- Batch normalization : $\sigma_{i,j} = \sigma_j$ [IS15]
- Layer normalization : $\sigma_{i,j} = \sigma_i$ [BKH16]

Batch norm depends on the batch hence is tricky to implement. Layer normalization is used in [VSP+17].



[IS15] S. Ioffe and C. Szegedy. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift* (Mar 2015), arXiv:1502.03167. Accessed on Nov 12, 2024.

[BKH16] J. L. Ba, J. R. Kiros and G. E. Hinton. *Layer Normalization* (Jul 2016), arXiv:1607.06450. Accessed on Nov 12, 2024.

[VSP+17] A. Vaswani, N. Shazeer, N. Parmar *et al.* *Attention Is All You Need*. In: *Advances in Neural Information Processing Systems*, Vol. 30 (Curran Associates, Inc., 2017). Accessed on Oct 11, 2024.

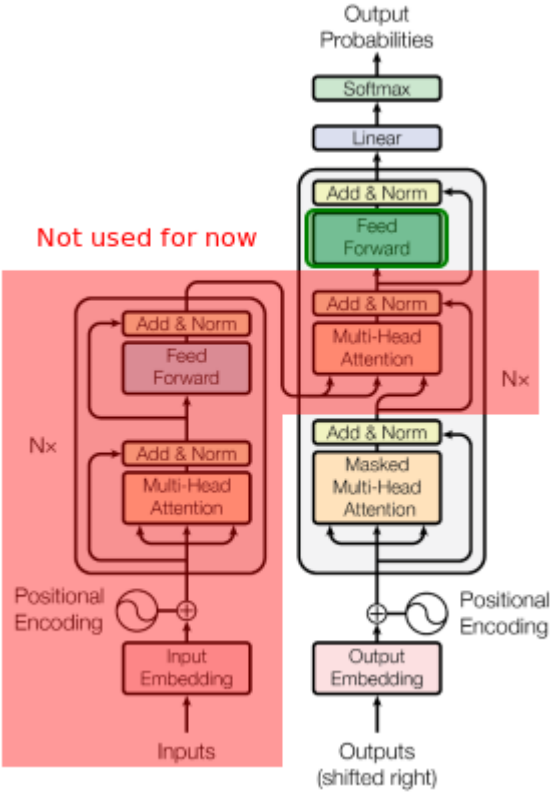
Feed-Forward network

Different weights $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{emb}}}$, $W_2 \in \mathbb{R}^{d_{\text{emb}} \times d_{\text{ff}}}$ for each layer:

$$x \mapsto W_2 \max(0, W_1 x + b_1) + b_2$$

Expansion factor $d_{\text{ff}}/d_{\text{emb}}$ is typically 4× like suggested in [VSP+17] (but not for Gemma)

Name	Ref	d_{emb}	d_{ff}
GPT-2	[RWCL19]	768	3072
Gemma	[TMHD24]	2048	32768
Gemma	[TMHD24]	3072	49152
Gemma-2	[TRPS24]	2304	18432
Gemma-2	[TRPS24]	3584	28672
Gemma-2	[TRPS24]	4608	73728
Llama-3			4096
base	[VSPU17]	512	2048
big	[VSPU17]	1024	4096



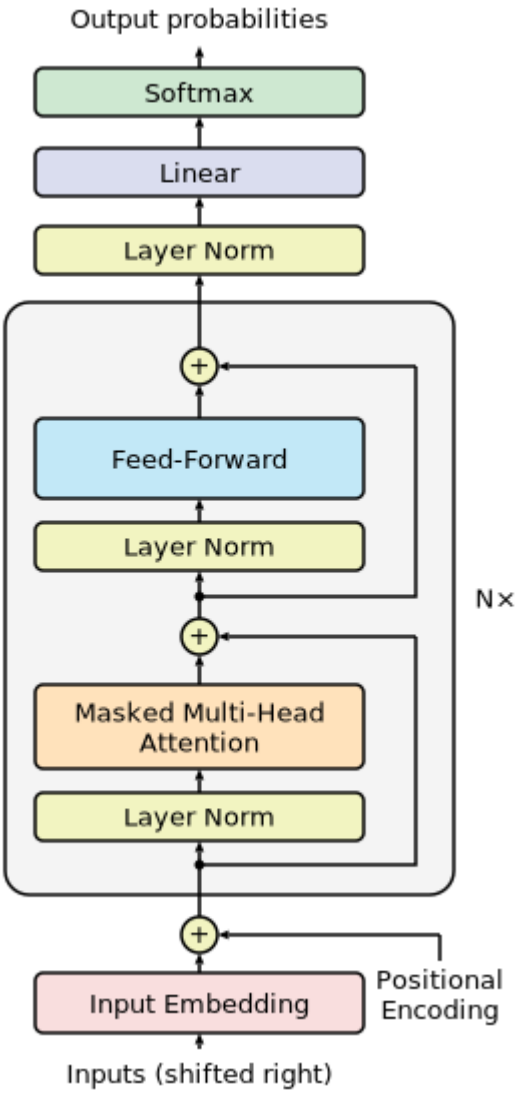
The feed-forward network is implemented **independently** for the output of each query so each query can be processed independently through each **layer**. The next layer allows each queries to then look at the results of the previous layer for **past** (because of the mask) queries.

Transformer variations

Pre-activation for residual neural networks introduced in [HZRS16] and used in GPT-2 [RWC+19]. See figure on the right.

Layer normalization is applied **before** attention and feed-forward, leaving an **identity** residual path. A final layer normalization follows the N blocks.

Rotary Positional Encoding [SLP+23] replaces $W^K(Cx_i + p_i)$ and $W^Q(Cx_i + p_i)$ by $R^i W^K C x_i$ and $R^i W^Q C x_i$ where R is a rotation matrix. Advantage : $\langle k_i, q_j \rangle$ contains R^{i-j} → **relative** difference of position.



[HZRS16] K. He, X. Zhang, S. Ren and J. Sun. *Identity Mappings in Deep Residual Networks*. In: *Computer Vision – ECCV 2016*, edited by B. Leibe, J. Matas, N. Sebe and M. Welling (Springer International Publishing, Cham, 2016); pp. 630–645.

[RWC+19] A. Radford, J. Wu, R. Child *et al.* *Language Models Are Unsupervised Multitask Learners* (2019). Accessed on Oct 23, 2024.

[SLP+23] J. Su, Y. Lu, S. Pan *et al.* *RoFormer: Enhanced Transformer with Rotary Position Embedding* (Nov 2023), arXiv:2104.09864. Accessed on Nov 12, 2024.

Cost of LLMs

▶ **What is the time complexity of inference with respect to d_{emb}, n_{voc}, n_{ctx}, d_{ff}, h and N ?**

▶ **How does the number of parameters of transformers compare with [BDV00] or RNNs for large n_{ctx} ?**

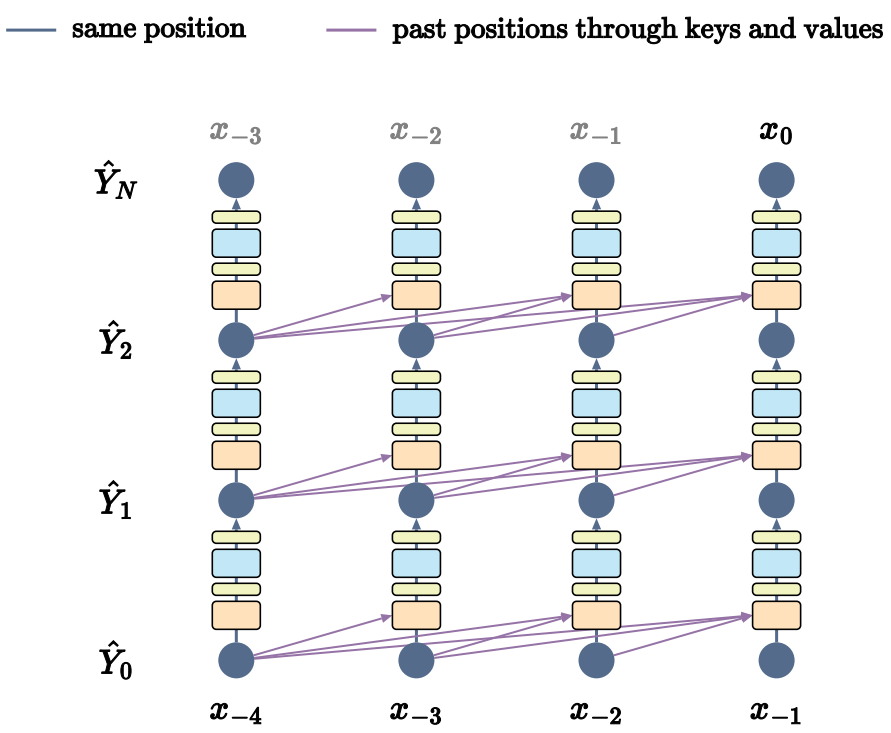
[BDV00] Y. Bengio, R. Ducharme and P. Vincent. *A Neural Probabilistic Language Model*. In: *Advances in Neural Information Processing Systems*, Vol. 13 (MIT Press, 2000). Accessed on Oct 11, 2024.

Dependencies between positions

Let $\hat{\boldsymbol{y}}_{i,j}$ be the column of $\hat{\boldsymbol{Y}}_i$ at the position of $\boldsymbol{x}_j$, with $\hat{\boldsymbol{Y}}_0 = \boldsymbol{C}\boldsymbol{X} + \boldsymbol{P}$. In layer $\boldsymbol{i}$, $\hat{\boldsymbol{y}}_{i,j}$ depends on

- $\hat{\boldsymbol{y}}_{i-1,j}$ for the query, the residual connection and the feed-forward network,
- $\hat{\boldsymbol{y}}_{i-1,k}$ for $\boldsymbol{k < j}$ **only** through their keys $\boldsymbol{W}_\ell^K \hat{\boldsymbol{y}}_{i-1,k}$ and values $\boldsymbol{W}_\ell^V \hat{\boldsymbol{y}}_{i-1,k}$ for each head $\boldsymbol{\ell}$,
- not on $\hat{\boldsymbol{y}}_{i-1,k}$ for $\boldsymbol{k > j}$ because of the mask.

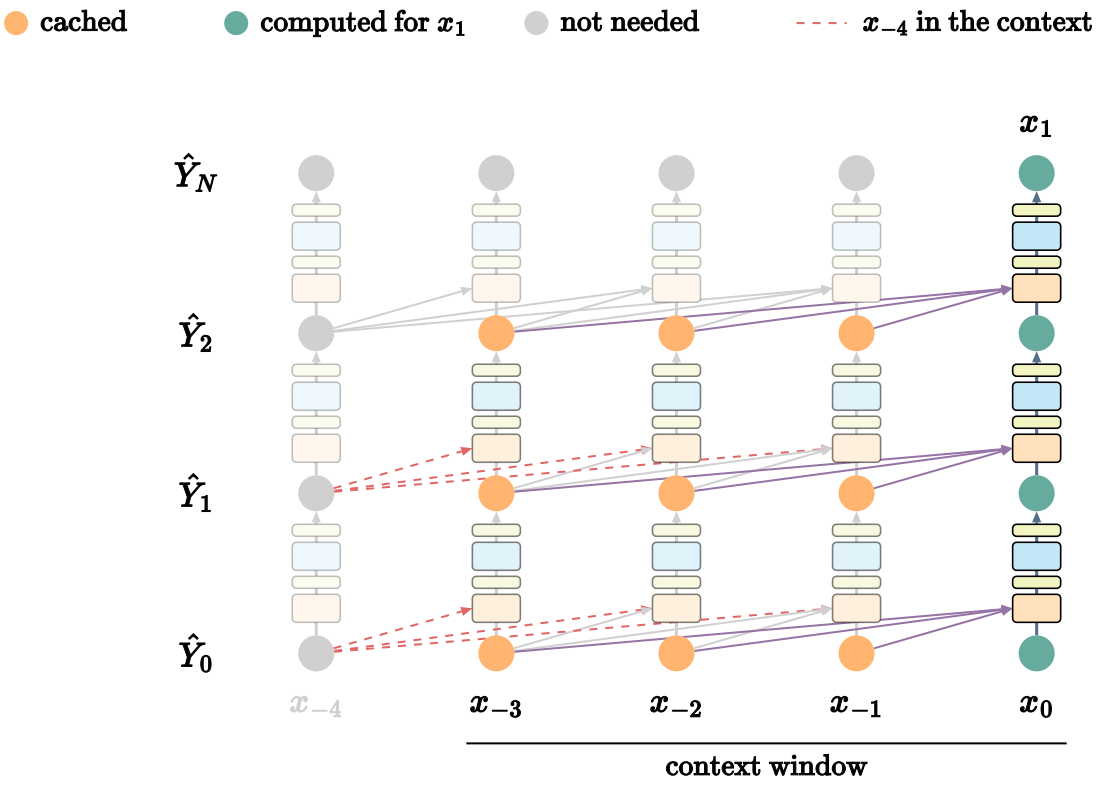
Given the keys and values of the past positions, the computation of $\hat{\boldsymbol{y}}_{i,j}$ is **independent** of the other positions.



▶ **What do we need to compute to predict x_0 ?**

Key-Value (KV) cache

Once x_0 is generated, the context window moves to $x_{-n_{\text{ctx}}+1}, \ldots, x_0$. 💡 For each layer $i < N$, the keys and values of $\hat{y}_{i,j}$ for $j < 0$ were already computed for the previous token, let's **cache** them! Since the positions are independent given these keys and values, we only need to compute the new column $\hat{y}_{i,0}$ of each layer i.



▶ **Is the KV cache equivalent to recomputing everything for the new context window ?**

▶ **Why does it still make sense ?**

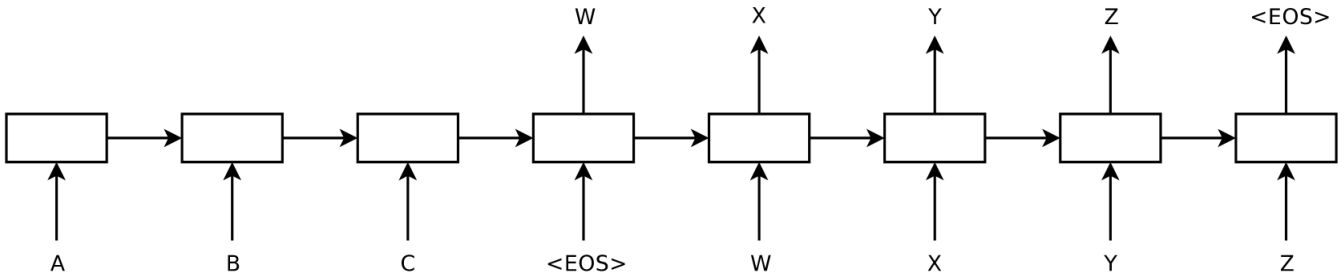
▶ **When would the KV cache not make sense ?**

▶ **What is the analogy with reverse-mode AD ? What is the cost ?**

Encoder-decoder transformer

Machine translation

- LSTM **encoder** → **context** → LSTM **decoder** [SVL14]. See [SVL14; Figure 1] below.
- Issue with *encoder bottleneck*. All information has to be summarized in the **context**.



[SVL14] I. Sutskever, O. Vinyals and Q. V. Le. *Sequence to Sequence Learning with Neural Networks*. In: *Proceedings of the 27th International Conference on Neural Information Processing Systems - Volume 2, NIPS'14* (MIT Press, Cambridge, MA, USA, Dec 2014); pp. 3104–3112. Accessed on Oct 23, 2024.

Cross-Attention

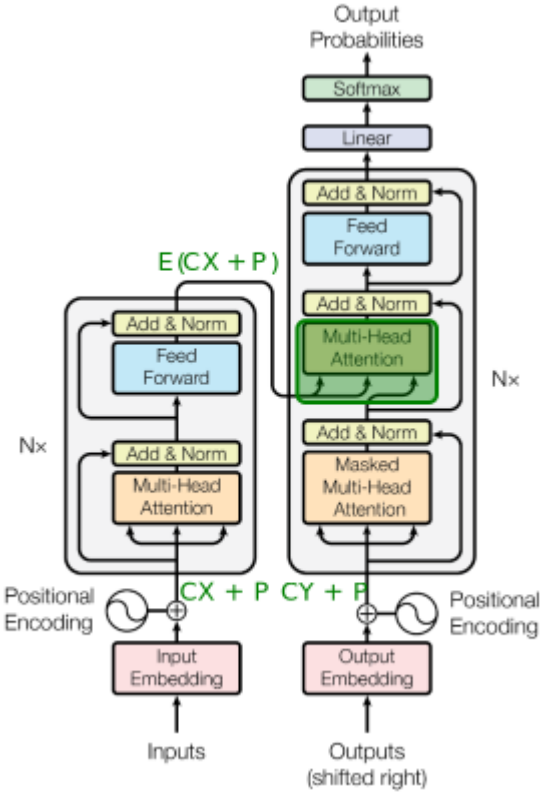
Cross-Attention between

- values and keys $\boldsymbol{E}(\boldsymbol{CX}+\boldsymbol{P})$ where $\boldsymbol{E}$ is the encoder, and $\boldsymbol{X}$ is the matrix of input tokens
- query $\boldsymbol{Q}$ depending on past output $\boldsymbol{Y}$ and number of layers already applied

$$\mathbf{MultiHead}(\boldsymbol{E}(\boldsymbol{CX}+\boldsymbol{P}), \boldsymbol{E}(\boldsymbol{CX}+\boldsymbol{P}), \boldsymbol{Q})$$

The embedding vectors $\boldsymbol{CX}$ take then different projections for value, key, query and also for different heads!

$$\mathbf{head}_j = \mathbf{Attention}(\boldsymbol{W}_j^V \boldsymbol{V}, \boldsymbol{W}_j^K \boldsymbol{K}, \boldsymbol{W}_j^Q \boldsymbol{Q})$$
$$\mathbf{where}\ \boldsymbol{V} = \boldsymbol{K} = \boldsymbol{E}(\boldsymbol{CX}+\boldsymbol{P})$$



Energy consumption of LLMs

Orders of magnitude

	Energy
10 W LED bulb for 1 hour	10 Wh = 36 kJ
Heating 1 L of water from 20 °C to 100 °C	≈ 93 Wh
Belgian reference household, per year	≈ 3,500 kWh
Data centres worldwide in 2024 (≈ 1.5% of world electricity)	≈ 415 TWh
Data centres worldwide in 2030 (IEA projection)	≈ 945 TWh

Source for data centres : IEA, *Energy and AI*, 2025.

As we will see, a **single** query costs a fraction of a Wh. The question is rather the number of queries and the energy needed to train the models.

Estimating the inference energy

A dense transformer with N parameters needs $\approx 2N$ FLOPs per generated token (one multiplication and one addition per weight), thanks to the KV cache. For a mixture of experts, N is the number of **active** parameters. An NVIDIA H100 performs $\approx 989 \cdot 10^{12}$ FLOP/s (dense BF16) at 700 W so $\approx 1.4 \cdot 10^{12}$ FLOP/J at **peak** utilization.

- Parameters N : 8B
- Generated tokens : 300
- GPU utilization : 0.1
- Overhead factor (host CPU, idle machines, cooling) : 1.7

Back-of-the-envelope estimate : 4.8e12 FLOP → 0.016 Wh per query → 5.8 seconds of a 10 W LED bulb.

This ignores the processing of the prompt and the attention cost which grows with the context length. The overhead factor 1.7 is the ratio between full stack and active TPU energy reported in

Elsworth et al., 2025.

▶ **Why is the GPU utilization so low when generating tokens ?**

Inference vs web search

Query	Wh / query	Scope	Source
Google search (2009)	0.3	Google servers, including building the index	Hölzle, 2009
Text generation, models ≤ 11B	≈ 0.05	Measured on 8×A100	Luccioni et al., 2024
Llama-3.1-8B, short / long prompt	0.05 / 0.44	Modelled, data centre	Jegham et al., 2025
Gemini Apps, median text prompt (May 2025)	0.24 (0.10 accelerators only)	Measured, full stack	Elsworth et al., 2025
GPT-4o, 500 output tokens	≈ 0.3	Estimate	You, Epoch AI, 2025
ChatGPT, average query	0.34	No methodology given	Altman, 2025
Llama-3.1-405B, short / long prompt	2.2 / 25	Modelled, data centre	Jegham et al., 2025
o3 (reasoning), short / long prompt	1.2 / 12	Modelled, data centre	Jegham et al., 2025
Image generation	≈ 2.9	Measured on 8×A100	Luccioni et al., 2024

- Small models and median prompts : **same order** as a (2009) web search.
- Large models, long contexts and reasoning : **1 to 2 orders of magnitude** more.
- Google reports a 33× reduction of the energy per prompt in 12 months Elsworth et al., 2025. There is no recent public figure for a web search.

Network and local device

Network : ≈ 0.06 kWh/GB in 2015, halving every 2 years Aslan et al., 2018. Extrapolating to ≈ 0.002 kWh/GB today, a 2.5 MB search result page (HTTP Archive, 2024) costs ≈ 0.005 Wh and a text answer of a few kB much less. Moreover, the power of the network is mostly **fixed** : it does not grow with the data volume so the **marginal** cost of a query is close to zero Mytton et al., 2024.

User device : A 20 W laptop used for 30 seconds consumes ≈ 0.17 Wh, the **same order** as the server side of the query! User devices account for more than half of the emissions of the ICT sector

Malmodin et al., 2024.

▶ **Is a small model running on a local GPU cheaper than a web search ?**

Training : compressing the web

Model	Energy	Scope	Source
GPT-3 (175B)	1.3 GWh	Estimate, V100, PUE 1.1	Patterson et al., 2021
BLOOM (176B)	0.43 GWh	Measured, A100, GPUs only	Luccioni et al., 2022
OLMo 2 (7B / 13B)	0.16 / 0.23 GWh	Measured, GPUs only	Morrison et al., 2025
Llama 3.1 (405B)	≤ 21.6 GWh	30.84M H100 hours × 700 W	Meta model card
GPT-4	$\approx 2 \cdot 10^{25}$ FLOP	Estimate	Epoch AI

- 21.6 GWh ≈ 6,000 Belgian households for a year.
- Experiments and development add a significant amount on top of the final training run : ≈ 50% for OLMo 2 Morrison et al., 2025.
- The CO_2 depends a lot on the electricity grid : BLOOM was trained on the French grid with 57 gCO_2/kWh Luccioni et al., 2022.

Training vs PageRank

Google search also needs an expensive precomputation : crawling the web, building the index and computing the PageRank.

- 1998 : "a PageRank for 26 million web pages can be computed in a few hours on a medium size workstation" Brin and Page, 1998.
- Today : the index covers "hundreds of billions of webpages" and is "well over 100,000,000 gigabytes" (Google).

Back-of-the-envelope : 10^{11} pages × 50 links × 2 FLOPs × 50 power iterations $\approx 5 \cdot 10^{14}$ FLOP for the PageRank, that is 10^{10} times less than GPT-4. However, PageRank is only a small part of the cost : crawling, parsing and storing the index dominate and there is **no public figure** for its energy. Note that LLMs also need the web to be crawled to obtain their training data.

▶ **After how many queries does the inference energy exceed the training energy ?**

Caveats

- **Scope** : the energy of the GPUs only, of the full servers or of the whole data centre differ by $\approx 2.4\times$ Elsworth et al., 2025.
- **Average vs tail** : company figures are medians or averages. Long contexts and reasoning models cost 10 to 100× more.
- **Dates** : the 0.3 Wh of a web search dates from 2009. Both web search and LLMs became much more efficient since then.
- **Fixed vs marginal** : idle servers and networks consume energy even without queries. BLOOM consumed ≈ 0.28 kWh every 10 minutes while idle Luccioni et al., 2022.
- **Rebound effect** : the energy per query decreases quickly but the number of queries grows even faster. The demand of data centres is expected to more than double by 2030 IEA, 2025.

Utils

```
using PlutoUI, DataFrames, PrettyTables, LinearAlgebra, Luxor, LaTeXStrings, MathTeXEngine, PlutoUI, PlutoUI.ExperimentalLayout, HypertextLiteral, PlutoTeachingTools
```

```
import DocumenterCitations, CSV, Logging
```

```
qa (generic function with 2 methods)
```

```
include("utils.jl")
```

```
biblio =
▸ CitationBibliography("/home/runner/work/LINMA2472/LINMA2472/Lectures/biblio.bib", AlphaSt
```

```
biblio = load_biblio!()
```

```
ⓘ Loading bibliography from `/home/runner/work/LINMA2472/LINMA2472/Lectures/biblio.bib`...
ⓘ Loading completed.
```

```
cite (generic function with 1 method)
```

```
cite(args...) = bibcite(biblio, args...)
```

```
bib (generic function with 1 method)
```

```
bib(args...) = bibrefs(biblio, args...)
```

```
draw_transformer (generic function with 2 methods)
```

```
function draw_transformer(decoder_only = true)
    scale(0.4, 0.4)
    Luxor.placeimage(readpng("images/transformer.png"), centered = true)
    if decoder_only
        sethue("red")
        setopacity(0.4)
        box(Point(-350, -160), Point(320, 20), :fill)
        box(Point(-350, 20), Point(0, 460), :fill)
        translate(Point(-170, -190))
        setopacity(1)
        fontsize(32)
        text("Not used for now", halign = :center)
    end
end
```

```
draw_preactivation_decoder (generic function with 1 method)
```

```
function draw_preactivation_decoder()
    x = -20
    function block(y, height, color, lines)
        sethue(color)
        box(Point(x, y), 164, height, 4, :fill)
        sethue("black")
        box(Point(x, y), 164, height, 4, :stroke)
        for (i, line) in enumerate(lines)
            text(
                line,
                Point(x, y + 15 * (i - (length(lines) + 1) / 2));
                halign = :center,
                valign = :middle,
            )
        end
    end
    function connection(y1, y2)
        arrow(Point(x, y1), Point(x, y2); linewidth = 1.5, arrowheadlength = 6)
    end
    function addition(y)
        sethue("#F2F4C1")
        circle(Point(x, y), 9, :fill)
        sethue("black")
        circle(Point(x, y), 9, :stroke)
        text("+", Point(x, y); halign = :center, valign = :middle)
    end
    function residual(y1, y2)
        circle(Point(x, y1), 2.5, :fill)
        poly([Point(x, y1), Point(88, y1), Point(88, y2)]; action = :stroke)
        arrow(Point(88, y2), Point(x + 9, y2); linewidth = 1.5, arrowheadlength = 6)
    end
    fontface("sans")
    fontsize(13)
    setline(1.5)
    sethue("#F4F4F4")
    box(Point(-5, 30), 224, 296, 12, :fill)
    sethue("black")
    box(Point(-5, 30), 224, 296, 12, :stroke)
    text("N×", Point(128, 30); halign = :center, valign = :middle)

    # Each residual path bypasses both the normalization and its sub-block.
    residual(163, 49)
    residual(29, -86)
    connection(189, 151)
    block(139, 24, "#F2F4C1", ["Layer Norm"])
    connection(127, 115)
    block(94, 42, "#FFE2BB", ["Masked Multi-Head", "Attention"])
    connection(73, 58)
```

```
    addition(49)
    connection(40, 16)
    block(4, 24, "#F2F4C1", ["Layer Norm"])
    connection(-8, -20)
    block(-40, 40, "#C2E8F7", ["Feed-Forward"])
    connection(-60, -77)
    addition(-86)

    # The final normalization is outside the repeated decoder block.
    connection(-95, -132)
    block(-144, 24, "#F2F4C1", ["Layer Norm"])
    connection(-156, -171)
    block(-183, 24, "#DADDEF", ["Linear"])
    connection(-195, -211)
    block(-223, 24, "#CEE8D1", ["Softmax"])
    connection(-235, -246)
    text("Output probabilities", Point(x, -260); halign = :center, valign = :middle)

    addition(198)
    connection(217, 207)
    block(232, 30, "#F9DEDE", ["Input Embedding"])
    connection(255, 247)
    text("Inputs (shifted right)", Point(x, 267); halign = :center, valign =
    :middle)
    line(Point(100, 211), Point(100, 198), :stroke)
    arrow(Point(100, 198), Point(x + 9, 198); linewidth = 1.5, arrowheadlength = 6)
    text("Positional", Point(100, 220); halign = :center, valign = :middle)
    text("Encoding", Point(100, 235); halign = :center, valign = :middle)
    return nothing
end
```

```
highlight (generic function with 1 method)
```

```
function highlight(a, b, c, d)
    sethue("green")
    setopacity(0.4)
    #box(Point(a, b), Point(c, d), :fill)
    polysmooth(box(Point(a, b), Point(c, d), vertices=true), 10, action = :fill)
    setopacity(1)
    polysmooth(box(Point(a, b), Point(c, d), vertices=true), 10, action = :stroke)
end
```

mathjax_figure

```
mathjax_figure(draw; width, height)
```

Luxor only draws the shapes, the labels are overlaid in HTML so that they are typeset by the MathJax of Pluto. `draw(label)` draws the shapes with the origin at the center and adds labels with `label(text, x, y; color, fontsize, halign)`.

```
"""
    mathjax_figure(draw; width, height)

Luxor only draws the shapes, the labels are overlaid in HTML so that
they are typeset by the MathJax of Pluto.
`draw(label)` draws the shapes with the origin at the center and adds labels
with `label(text, x, y; color, fontsize, halign)`.
"""
function mathjax_figure(draw; width, height)
    labels = []
    function label(text, x, y; color = "black", fontsize = 14, halign = :center)
        dx = halign == :left ? "0" : "-50%"
        push!(labels, @htl("""<span class="tex" style="position: absolute; left:
        $(x + width / 2)px; top: $(y + height / 2)px; transform: translate($dx,
        -50%); white-space: nowrap; color: $color; font-size:
        $(fontsize)px;">\$$text\$</span>"""))
    end
    Luxor.Drawing(width, height, :svg)
    Luxor.origin()
    Luxor.background("white")
    draw(label)
    Luxor.finish()
    svg = Luxor.svgstring()
    @htl("""<div style="position: relative; width: $(width)px; height:
    $(height)px;">
    <div style="position: absolute; inset: 0;">$(HTML(svg))</div>
    $labels
    </div>""")
end
```

transformer_graph

```
transformer_graph(; kv_cache = false, needed = false, n_ctx = 4, N = 3)
```

Draw the dependencies between the columns of $\hat{Y}_0, \ldots, \hat{Y}_N$. If `needed` is `true`, the columns that are not needed to predict x_0 are greyed out. If `kv_cache` is `true`, show the computation of the next token with the KV cache.

```
"""
    transformer_graph(; kv_cache = false, needed = false, n_ctx = 4, N = 3)

Draw the dependencies between the columns of ``\\hat{Y}_0, \\ldots, \\hat{Y}_N``.
If `needed` is `true`, the columns that are not needed to predict ``x_0`` are greyed out.
If `kv_cache` is `true`, show the computation of the next token with the KV cache.
"""
function transformer_graph(; kv_cache = false, needed = false, n_ctx = 4, N = 3, width = 640, height = 420)
    mathjax_figure(; width, height) do label
        colors = (computed = "#546b8c", kv = "#9673a6", cached = "#ffb570", new = "#67ab9f", unneeded = "#d0d0d0", extra = "#e06666")
        js = kv_cache ? (-n_ctx:0) : (-n_ctx:-1)
        Δx = 90
        Δy = 80
        X(j) = (j - (first(js) + last(js)) / 2) * Δx + 50
        Y(i) = (N / 2 - i) * Δy
        radius = 9
        # Blocks of a layer from bottom to top with the colors of the figure
        # of the paper: attention, add & norm, feed-forward, add & norm
        blocks = [(14, "#FFE2BB"), (6, "#F2F4C1"), (14, "#C2E8F7"), (6, "#F2F4C1")]
        block_width = 24
        gap = 3
        stack_height = sum(first, blocks) + gap * (length(blocks) - 1)
        stack_bottom(i) = (Y(i - 1) + Y(i) + stack_height) / 2
        attention(i, j) = Luxor.Point(X(j) - block_width / 2, stack_bottom(i) - first(blocks[1]) / 2)
        in_window(j) = !kv_cache || j > -n_ctx
        function status(i, j)
            if kv_cache
                j == 0 && return :new
                (i == N || !in_window(j)) && return :unneeded
                return :cached
            end
            needed && i == N && j != -1 && return :unneeded
            return :computed
        end
```

```
        # Edges from position `k` in layer `i - 1` to position `j` in layer `i`
        for i in 1:N, j in js, k in js
            k <= j || continue
            s = status(i, j)
            if s == :new
                in_window(k) || continue
                color = k == j ? colors.computed : colors.kv
            elseif kv_cache && s == :cached && !in_window(k)
                # The cached values still have `x_{n_ctx}` in their context
                color = colors.extra
            elseif s == :unneeded || (kv_cache && s == :cached)
                color = colors.unneeded
            else
                color = k == j ? colors.computed : colors.kv
            end
            p = Luxor.Point(X(k), Y(i - 1))
            # The past positions only enter through the attention
            q = k == j ? Luxor.Point(X(j), Y(i) + radius) : attention(i, j)
            d = (q - p) / Luxor.distance(p, q)
            Luxor.sethue(color)
            Luxor.setdash(color == colors.extra ? "dash" : "solid")
            Luxor.arrow(p + radius * d, q; linewidth = k == j ? 1.5 : 1,
            arrowheadlength = 6)
        end
        Luxor.setdash("solid")
        for i in 1:N, j in js
            s = status(i, j)
            opacity = s in (:computed, :new) ? 1 : s == :cached ? 0.5 : 0.25
            y = stack_bottom(i)
            for (h, color) in blocks
                center = Luxor.Point(X(j), y - h / 2)
                # Opaque white background so that the arrow behind is hidden
                # when the block is drawn with some transparency
                Luxor.setopacity(1)
                Luxor.sethue("white")
                Luxor.box(center, block_width, h, 2, :fill)
                Luxor.setopacity(opacity)
                Luxor.sethue(color)
                Luxor.box(center, block_width, h, 2, :fill)
                Luxor.sethue("black")
                Luxor.setline(0.8)
                Luxor.box(center, block_width, h, 2, :stroke)
                y -= h + gap
            end
        end
        Luxor.setopacity(1)
        Luxor.setline(1.5)
        for i in 0:N, j in js
            s = status(i, j)
            Luxor.sethue(colors[s])
```

```
            Luxor.circle(Luxor.Point(X(j), Y(i)), radius, :fill)
        end
        for i in 0:N
            label(i == N ? "\\hat{Y}_N" : "\\hat{Y}_$i", X(first(js)) - 60, Y(i))
        end
        for j in js
            label("x_{$j}", X(j), Y(0) + 25; color = in_window(j) ? "black" :
            colors.unneeded)
            s = status(N, j)
            if s in (:computed, :new)
                label("x_{$(j + 1)}", X(j), Y(N) - 25; color = j == last(js) ?
                "black" : "gray")
            end
        end
        if kv_cache
            y = Y(0) + 45
            Luxor.sethue("black")
            Luxor.setline(1)
            Luxor.line(Luxor.Point(X(-n_ctx + 1) - 15, y), Luxor.Point(X(0) + 15,
            y), :stroke)
            label("\\text{context window}", (X(-n_ctx + 1) + X(0)) / 2, y + 12;
            fontsize = 12)
            y = -height / 2 + 15
            for (x, s, text) in [(-280, :cached, "\\text{cached}"), (-170, :new,
            "\\text{computed for } x_1"), (-20, :unneeded, "\\text{not needed}")]
                Luxor.sethue(colors[s])
                Luxor.circle(Luxor.Point(x, y), 6, :fill)
                label(text, x + 12, y; fontsize = 12, halign = :left)
            end
            Luxor.sethue(colors.extra)
            Luxor.setdash("dash")
            Luxor.line(Luxor.Point(110, y), Luxor.Point(135, y), :stroke)
            Luxor.setdash("solid")
            label("x_{$(-n_ctx)} \\text{ in the context}", 143, y; fontsize = 12,
            halign = :left)
        else
            y = -height / 2 + 15
            for (x, color, text) in [(-200, colors.computed, "same position"),
            (-40, colors.kv, "past positions through keys and values")]
                Luxor.sethue(color)
                Luxor.line(Luxor.Point(x, y), Luxor.Point(x + 25, y), :stroke)
                label("\\text{$text}", x + 33, y; fontsize = 12, halign = :left)
            end
        end
    end
```

```
struct BPE
    text::String
    pairs::Dict{Tuple{Char,Char},Char}
end
```

```
add_pair (generic function with 1 method)
```

```
function add_pair(bpe::BPE, subs)
    pairs = copy(bpe.pairs)
    push!(pairs, subs)
    return BPE(replace(bpe.text, prod(subs.first) => subs.second), pairs)
end
```

```
pair_stats (generic function with 1 method)
```

```
function pair_stats(text::String)
    stats = Dict{Tuple{Char,Char},Int}()
    for i in eachindex(text)
        j = nextind(text, i)
        if j > lastindex(text)
            break
        end
        a = text[i]
        b = text[j]
        stats[(a, b)] = get(stats, (a, b), 0) + 1
    end
    return stats
end
```

```
substitute (generic function with 1 method)
```

```
function substitute(text::String, pair::Tuple{Char,Char})
    new_char = min('Z' + 1, minimum(text)) - 1
    return replace(text, prod(pair.first) => pair.second)
end
```

```
new_token (generic function with 1 method)
```

```
new_token(text::String) = new_token(BPE(text, Dict()))
```

```
new_token (generic function with 2 methods)
```

```
function new_token(bpe::BPE)
    stats = pair_stats(bpe.text)
    pair = findmax(stats)[2]
    new_char = min('Z' + 1, minimum(bpe.text)) - 1
    return add_pair(bpe, pair => new_char)
end
```

llms =

	Name	Num params	Ref	``n_\tex
1	"Gemini-1.5"	missing	"[TGLB24]"	"256k"
2	"Gemini-1"	"1.[8B/3.25B](https://storage.googleapis.	"[TABA24]"	"256k"
3	"Gemma-2"	"27B"	"[TRPS24]"	"256k"
4	"Gemma-2"	"9B"	"[TRPS24]"	"256k"
5	"Gemma-2"	"2B"	"[TRPS24]"	"256k"
6	"Gemma"	"7B"	"[TMHD24]"	"256k"
7	"Gemma"	"2B"	"[TMHD24]"	"256k"
8	"GPT-2"	"1.5B"	"[RWCL19]"	"[50k](https://githu
9	"Llama-2"	"7B"	"[TMSA23]"	"[32k](https://githu
10	"GPT-4o"	missing	missing	"[200k](https://gith
⋮	more			
19	"big"	missing	"[VSPU17]"	"37k"

```
llms = load_llms()
```

```
load_llms (generic function with 1 method)
```

```
function load_llms()
    llms = DataFrame(CSV.File("llms.csv"))
    rename!(llms, "Embedding dimension" => "``d_\\text{emb}``")
    rename!(llms, "Vocabulary size" => "``n_\\text{voc}``")
    rename!(llms, "Context window" => "``n_\\text{ctx}``")
    rename!(llms, "Feed-Forward hidden dimension" => "``d_\\text{ff}``")
    return llms
end
```

```
▸ ["Name", "Num params", "Ref", "``n_\\text{voc}``", "``d_\\text{emb}``", "``n_\\text{ctx}`
```

```
names(llms)
```

```
table (generic function with 1 method)
```

```
function table(df; mandatory_columns = String[], included_columns = nothing)
    for col in mandatory_columns
        df = df[(!ismissing).(df[!, col]), :]
    end
    if !isnothing(included_columns)
        df = unique(df[!, included_columns])
    end
    Markdown.parse(pretty_table(
        String,
        sort(df),
        backend = :markdown,
        column_labels = names(df),
        allow_markdown_in_cells = true,
        formatters = [(v, _, _) -> ismissing(v) ? "" : v],
    ))
end
```